Algoritmusok és adatszerkeztek II.

11. gyakorlat

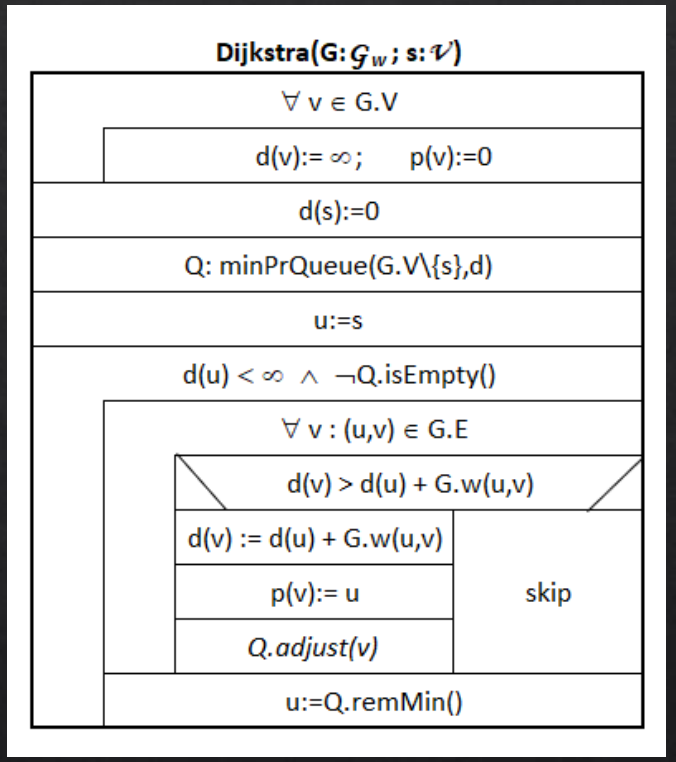
Tartalom:

Legrövidebb utak egy forrásból 2.
Legrövidebb utak minden csúcspárra

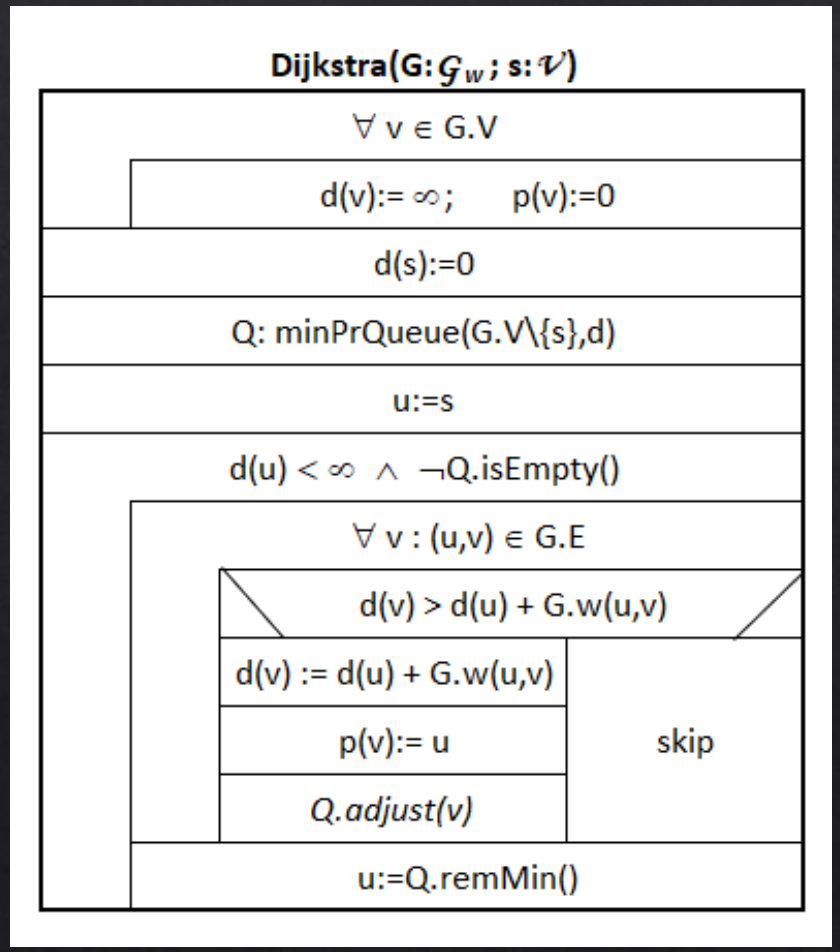
- Szorgalmi házi feladat megoldása
- Bellman-Ford algoritmus
- Sor alapú Bellman-Ford algoritmus
- Sor alapú Bellman-Ford algoritmus lejátszása
- Negatív kör észlelése
- Sor alapú Bellman-Ford implementálása csúcsmátrixszal ábrázolt gráfon
- Legrövidebb utak minden csúcspárra
- Floyd-Warshall algoritmus
- Floyd Warshall algoritmus lejátszása

Szorgalmi házi feladat megoldása

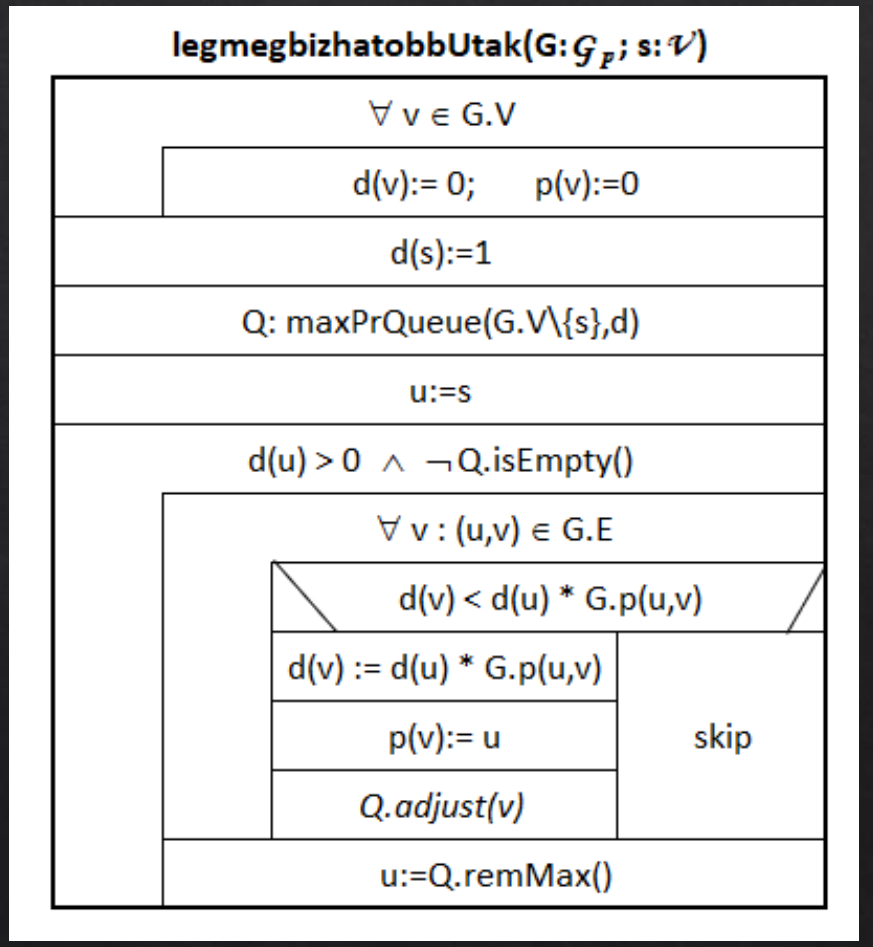
- A Dijkstra algoritmus átírása a legmegbízhatóbb útvonalak meghatározására.
- Módosítsuk úgy az algoritmust, hogy az éleken megadott p(u,v) valószínűséggel számolva, egy adott s csúcsból meghatározza az elérhető csúcsokhoz vezető legbiztonságosabb utakat.
- $G_p = (V,E,p)$, ahol $p: G.E \rightarrow (0,1]$, azaz $\forall (u,v) \in G.E: 0<p(u,v)\leq 1$
- Egy $s,u_1,u_2,\ldots u_m,v$ út megbízhatóságát az úton lévő élek megbízhatóságának szorzatával adhatjuk meg: $p(s,u_1)*p(u_1,u_2) * \ \ldots \ *p(u_m,v)$



Megoldás

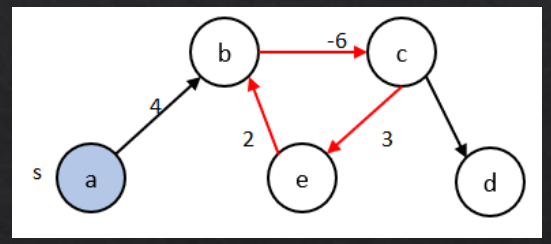

Dijkstra(G:$\mathcal{G}_w$; s:$\mathcal{V}$)
∀ v ∈ G.V
d(v):= ∞ ; p(v):=0
d(s):=0
Q: minPrQueue(G.V\{s},d)
u:=s
d(u) < ∞ ∧ ¬Q.isEmpty()
∀ v : (u,v) ∈ G.E
d(v) > d(u) + G.w(u,v)
d(v) := d(u) + G.w(u,v)
p(v):= u
Q.adjust(v)
skip
u:=Q.remMin()

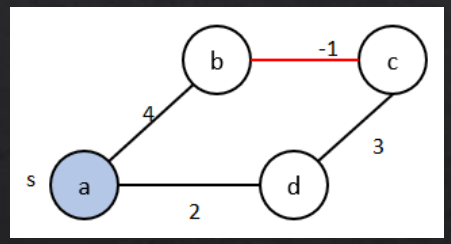



legmegbizhatobbUtak(G:$\mathcal{G}_p$; s:$\mathcal{V}$)
∀ v ∈ G.V
d(v):= 0; p(v):=0
d(s):=1
Q: maxPrQueue(G.V\{s},d)
u:=s
d(u) > 0 ∧ ¬Q.isEmpty()
∀ v : (u,v) ∈ G.E
d(v) < d(u) * G.p(u,v)
d(v) := d(u) * G.p(u,v)
p(v):= u
Q.adjust(v)
skip
u:=Q.remMax()

Bellman-Ford algoritmus
első változat

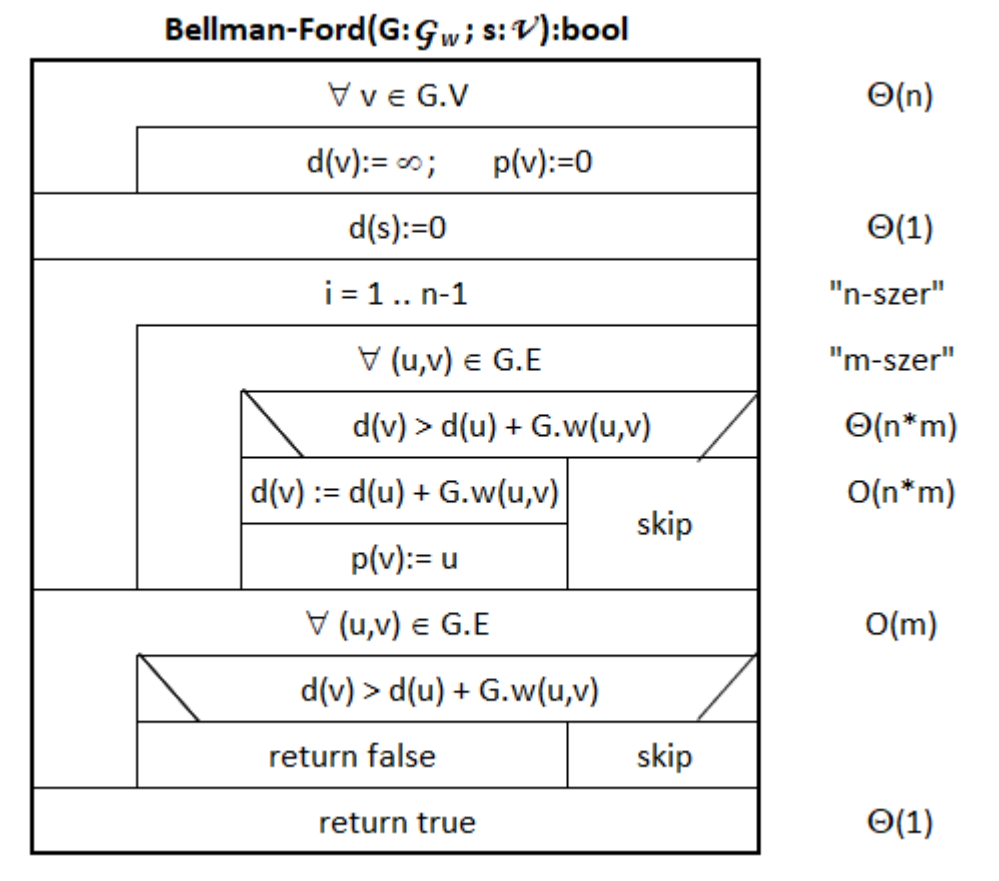
- Legrövidebb utak egy forrásból probléma megoldását adja, abban az esetben (is), amikor irányított a gráf, és negatív költségű élt is tartalmaz.
- Nem lehet olyan negatív összköltségű kör a gráfban, amely a start csúcsból elérhető, ezért ha irányítatlan a gráf, nem lehet a start csúcsból elérhető élek között negatív.





- Ha van ilyen kör a gráfban, azt az algoritmus jelzi.
- Használható nem negatív élsúlyú irányított/irányítatlan gráf esetén is, de költségesebb a Dijkstra algoritmusnál.

Az algoritmus, és műveletigénye

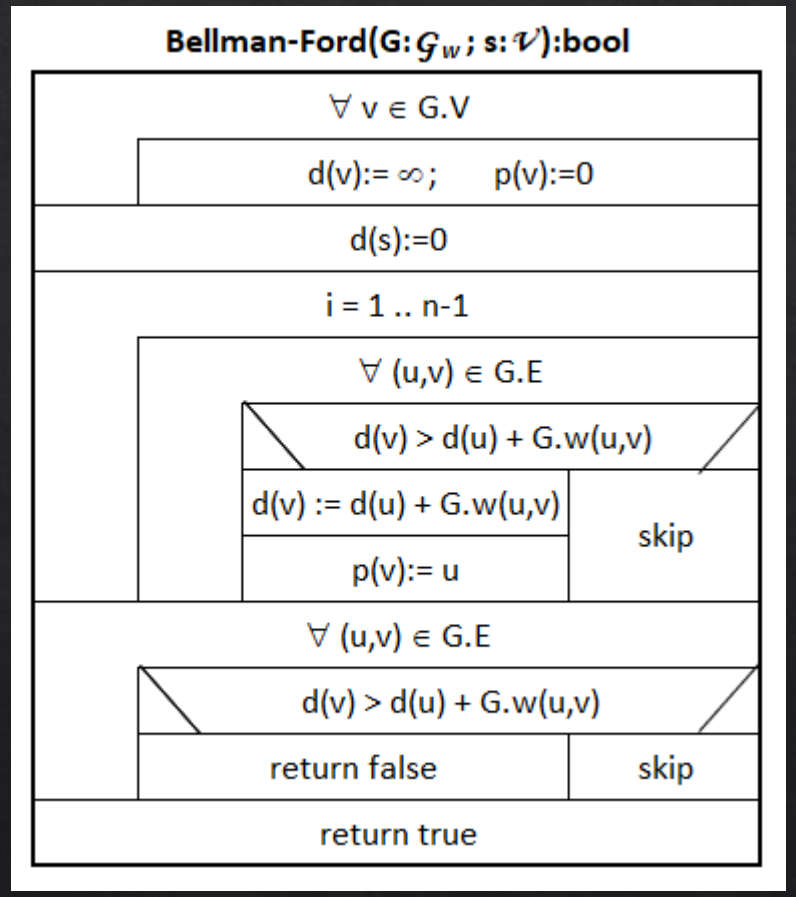


Első változat, nem a jegyzetben szereplő!

- Fő ciklusa n-1 menetet végez, a menetek invariáns tulajdonsága:
  - i-edik menetben a start csúcsból induló, legfeljebb i darab élt tartalmazó megoldást állítja elő.
- Mivel n csúcs esetén az egyszerű utak legfeljebb n-1 élt tartalmazhatnak, az n-1-edik végére előáll minden elérhető csúcshoz a megoldás, ha létezik.
- Még egy plusz menetet végezve, ha akkor is csökken valamely csúcs d() értéke, akkor negatív kört tartalmaz a gráf, ezt hamis visszatérési értékkel jelzi.
- Műveletigénye $\Theta(n*m)$:
  - Ritka gráf esetén: $\Theta(n^2)$ –tel becsülhető,
  - Sűrű gráf esetén viszont $\Theta(n^3)$ –öt kapunk.

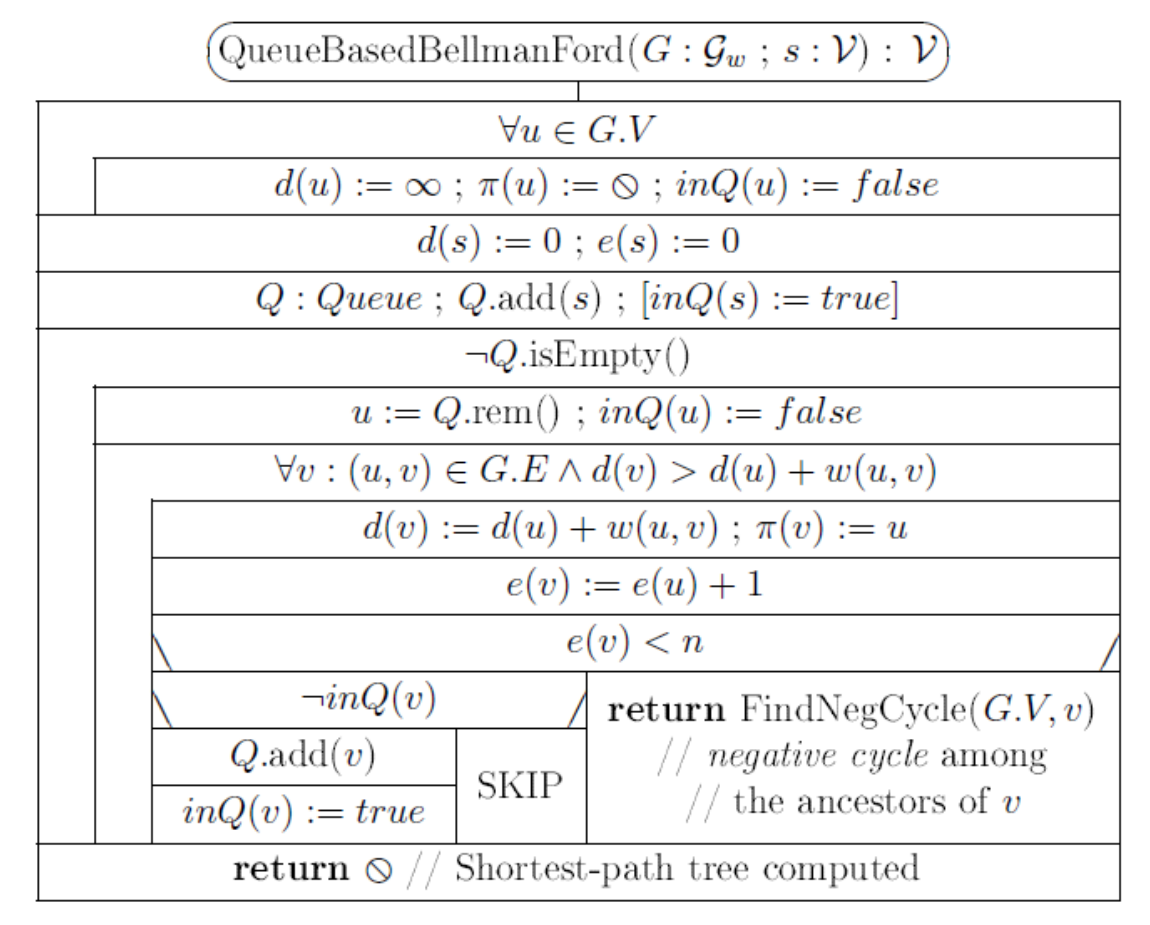
A sor alapú Bellman-Ford algoritmus, hatékonyabbá teszi ezt az első változatot

- Az algoritmust hatékonyabbá tehetjük, ha a belső ciklus nem vizsgálja meg a gráf összes élét!
- Elegendő azoknál a csúcsoknál vizsgálni az éleket, amelyekhez az előző menetben javító élt találtunk.
- Azokat a csúcsokat, amelyeknek egy menetben csökken a d() értéke (ez egy meneten belül akár többször is előfordulhat), berakjuk egy sorba.
- A következő menetben csak a sorban lévő csúcsokkal végzünk közelítést.
- Egy segéd tömb szolgál arra, hogy jelezze, mely csúcsok vannak épp a sorban, hogy azokat nehogy ismételten berakjuk.



Első változat, nem a jegyzetben szereplő!

Sor alapú Bellman-Ford algoritmus

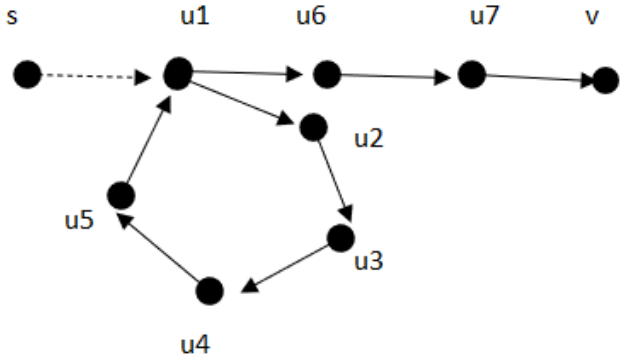


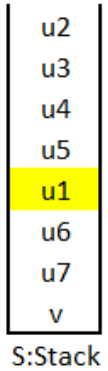
- A csúcsok inQ() címkéje jelzi, hogy a csúcs éppen a sorban van vagy nincs.
- A csúcsok e() címkéje tartja nyilván, hogy hány élt tartalmaz a csúcshoz vezető minimális költségű út.
  - Amíg ez kisebb, mint *n*, addig nincs baj.
  - Ha eléri az *n*-et, akkor negatív kört találtunk a gráfban.
  - Ekkor egy segéd eljárással a negatív kör egy csúcsa felderíthető.
- Műveletigény: MT(n,m) = O(n*m)

Sor alapú Bellman-Ford algoritmus, negatív kör

```
FindNegCycle(V : V{} ; v : V) : V
// Find a vertex of a negative cycle among the ancestors of v
∀u ∈ V
    B(u) := false
B(v) := true ; u := π(v)
¬B(u)
    B(u) := true ; u := π(u)
return u // u is a vertex of a negative cycle
```

- A negatív összköltségű kör egy csúcsát megkereső segéd eljárás.
- Egy olyan csúcsból visszafelé haladva, melynek csökkent a d() címkéje, és a hozzá vezető út hossza n, visszafelé lépkedve a szülőkön, megtalálhatjuk a kör egy csúcsát.
- Ha közben egy verembe rakjuk a csúcsokat, a kört a veremből ki is tudjuk írni!





B(u1)=true!

megtaláltuk a kört

Ha u az aktuális csúcs (u=u1), akkor a kör kiírása:

```
write(u)
    S.top() ≠ u
        write(S.pop())
```

Gyakorlati tapasztalat a futási idővel kapcsolatosan

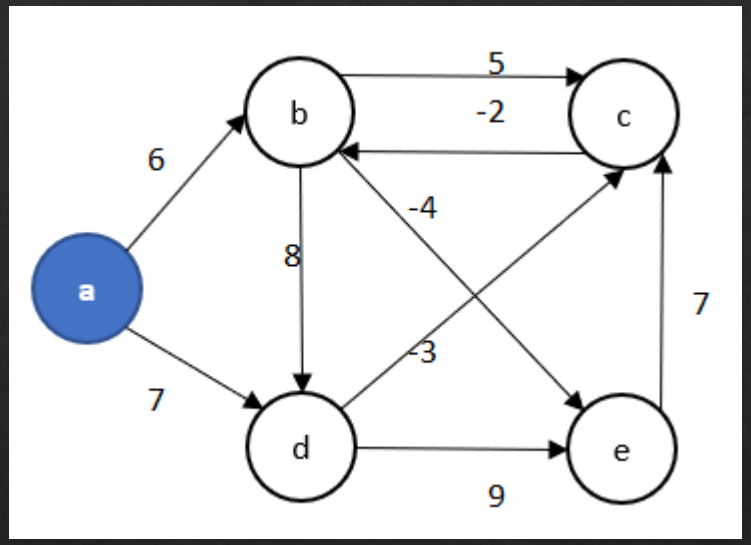
- Idézet a jegyzetből:
- *Az elméleti műveletigény becslés [6] meglehetősen rossz hatékonyságot sugall.*
- *Gyakorlati tesztek pozitítív élsúlyú, véletlenszerűen generált, nagyméretű, s-ből összefüggő ritka gráfokra ($m \in O(n)$) statisztikusan $\Theta(n)$ átlagos műveletigényt adtak, szomszédossági listás gráfábrázolás esetén.*
- *Ez viszont aszimptotikusan az elméleti minimummal egyezik. A hálózatok jelentős része nagyméretű, ritka gráffal modellezhető. Nem véletlen, hogy hálózatokon való útkeresésnél alkalmazzák is ezt a viszonylag egyszerű, de általános algoritmust.*

- Ez pedig azt jelenti, hogy ritka gráfokon, pozitív élsúly esetén is érdemesebb ezt az algoritmust használni, mint a Dijkstra algoritmust.
- Élek számának becslése a kísérletben: $m \leq k*n$, $k \approx 4$, előre adott konstans.

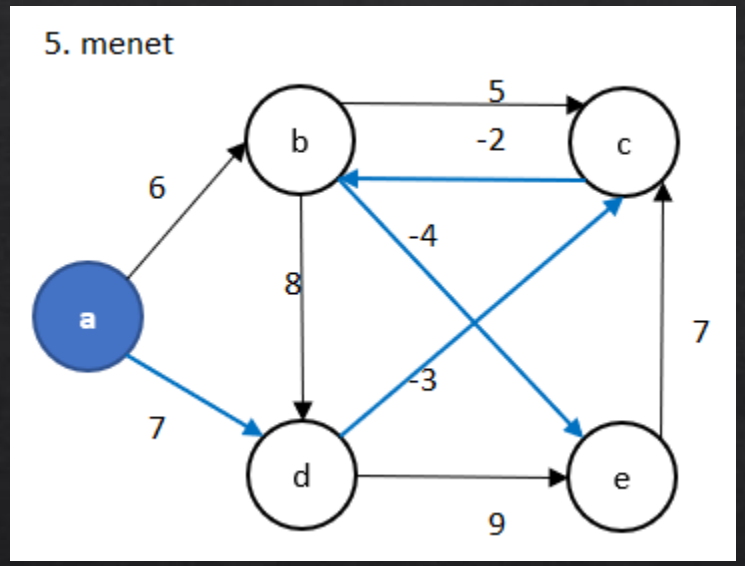
[6] Tarjan, Robert Endre, Data Structures and Network Algorithms, *CBMS-NSF Regional Conference Series in Applied Mathematics*, 1987.

Bellman-Ford algoritmus lejátszása

- Szemléltessük az algoritmus működését, az itt látható gráfon,
- Majd oldjuk meg a gyakorló feladatot.
- BF_peldak.xlsx



Az algoritmussal kapott megoldás, és a működést szemléltető táblázat



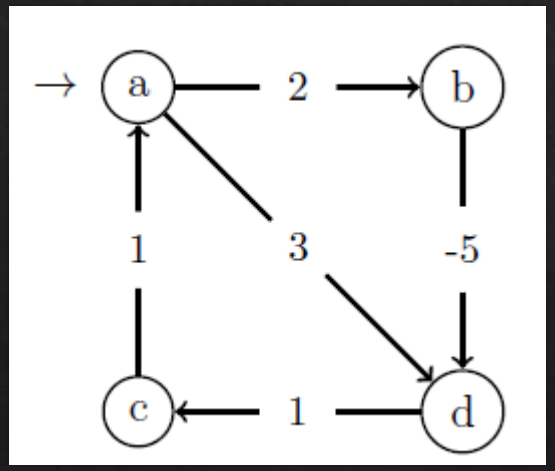
d(); e() változásai					kiterjesztés:	Q	p() változásai					menet
a	b	c	d	e	d();e()		a	b	c	d	e	
0;0	∞	∞	∞	∞		<a>	0	0	0	0	0	
	6;1		7;1		a: 0;0	<b,d>		a		a		1
		11;2		2;2	b: 6;1	<d,c,e>			b		b	2
		4;2			d: 7;1	<c,e>			d			2
	2;3				c: 4;2	<e,b>		c				3
					e: 2;2	<b>						3
				-2;4	b: 2;3	<e>					b	4
					e: -2;4	<>						5

0;0	2;3	4;2	7;1	-2;4			0	c	d	a	b	

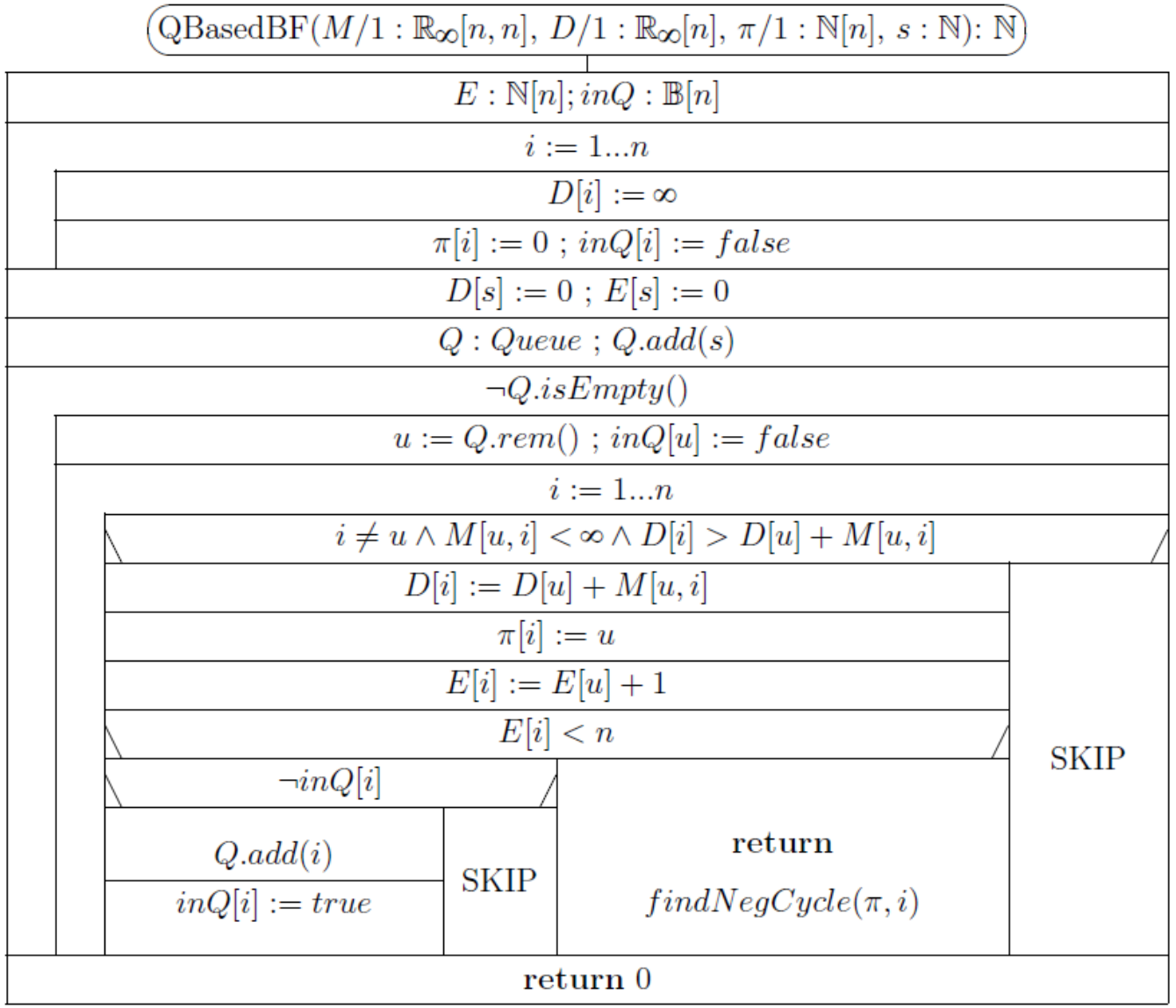
Negatív kör felfedezése

QueueBasedBellmanFord($G : \mathcal{G}_w$; $s : \mathcal{V}$) : $\mathcal{V}$

- $\forall u \in G.V$
  - $d(u) := \infty$; $\pi(u) := \oslash$; $inQ(u) := false$
- $d(s) := 0$; $e(s) := 0$
- $Q : Queue$; Q.add(s) ; $[inQ(s) := true]$
- $\neg Q$.isEmpty()
  - $u := Q$.rem() ; $inQ(u) := false$
  - $\forall v : (u, v) \in G.E \wedge d(v) > d(u) + w(u, v)$
    - $d(v) := d(u) + w(u, v)$; $\pi(v) := u$
    - $e(v) := e(u) + 1$
    - $e(v) < n$
      - $\neg inQ(v)$
        - Q.add(v)
        - $inQ(v) := true$
      - SKIP
      - **return** FindNegCycle($G.V, v$) // *negative cycle* among // the ancestors of v
- **return** $\oslash$ // Shortest-path tree computed



$d; e$ változásai				kiterjesztés:d;e	Q	π változásai				menet
a	b	c	d			a	b	c	d	
0; 0	∞	∞	∞	–	⟨ a ⟩	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	–
	2;1		3;1	a : 0;0	⟨ b, d ⟩		a		a	0
			-3;2	b : 2;1	⟨ d ⟩				b	1
		-2;3		d : -3;2	⟨ c ⟩			d		1
-1; ④				c : -2;3	⟨ a ⟩	c				2



- Feladat: Írjuk fel az algoritmust csúcsmátrixszal ábrázolt gráfra, a csúcsok cimkéit tömbökkel megvaósítva.
- A csúcsokat 1..n sorszámokkal azonosítjuk.

Legrövidebb utak minden csúcspárra

- Feladat: Adott egy élsúlyozott gráf, keressük minden csúcspárra a két csúcs között lévő legrövidebb utat.
- Ilyen feladat adódik például egy autóstérkép készítésekor is (városok közötti távolság táblázat).
- Irányított gráf esetén megengedünk negatív éleket, negatív összsúlyú köröket azonban nem, irányítatlan esetben negatív éleket sem engedünk meg.
- A feladatot megoldhatjuk a Dijkstra vagy a Bellman-Ford algoritmussal is, vizsgáljuk meg, hogy ez milyen futási idő költséggel járna.
- A megoldást úgy kaphatjuk meg, hogy minden csúcsból, elindítjuk az algoritmusokat.

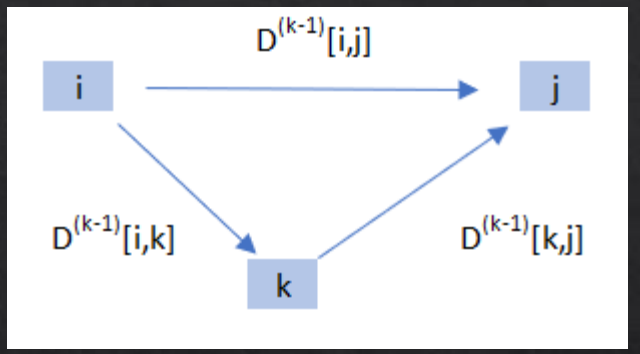
- Dijkstra algoritmus:
  - Ritka gráfon, kupacos minQueue esetén, egy csúcsból elindítva $MT(n,m)=O((n+m)*\log n)$
  - Minden csúcsból elindítva: $O(n*(n+m)*\log n)$, feltéve, hogy $m \in \Theta(n)$ $MT(n,m)=O(n^2*\log n)$
  - Sűrű gráfon, rendezetlen tömböt használva minQueue helyett, egy csúcsból elindítva: $MT(n,m)= \Theta(n^2)$ feltéve, hogy $m \in \Theta(n^2)$
  - Minden csúcsból elindítva: $MT(n,m)= \Theta(n^3)$
- Bellman-Ford algoritmus:
  - Egy csúcsra elindítva: $MT(n,m)=O(n*m)$
  - Ritka gráfon, minden csúcsból elindítva $MT(n,m)=O(n^2*m)=O(n^3)$, $m \in \Theta(n)$ esetén.
  - Sűrű gráfra: $MT(n,m)=O(n^2*m)=O(n^4)$, mivel $m \in \Theta(n^2)$ ilyenkor.
- Floyd-Warshall algoritmus: $\Theta(n^3)$
  - közelítő ciklusa annyira egyszerű, hogy hatékonyabb, mint például a Dijkstra algoritmus, vagy a sor alapú Belmann Ford.

Floyd-Warshall algoritmus

- Tegyük fel, hogy G gráf csúcsmátrixos reprezentációval adott.
- Az egyes csúcsok közötti távolságok számításához egy D-vel jelölt mátrixot fogunk használni, D[i,j] az i és j csúcs közti optimális út hosszát tartalmazza.
- Egy Π mátrixban a csúcsokhoz vezető optimális utakat tartjuk nyiltván: a közvetlen megelőző csúcsokat tároljuk. Π [i,j] az i és j csúcs között vezető optimális úton j csúcs közvetlen megelőzőjét tartalmazza.
- Az optimális utakat úgynevezett *belső csúcsok* mentén keressük a *dinamikus programozás* tervezési módszert használva.
- Egy $v_1,v_2,\ldots,v_m$ út belső csúcsai: $v_2,..v_{m-1}$

Floyd-Warshall algoritmus

- Az algoritmus n lépésben határozza meg a csúcspárok közti legrövidebb utakat.
- A k-adik lépése után a $D^{(k)}[i,j]$ az i és j csúcsok között lévő optimális út hosszát tartalmazza, úgy, hogy, a belső csúcsok címkéje legfeljebb k, a k címkéjű csúcs vagy szerepel az úton vagy nem, attól függően, hogy az út ettől optimálisabb lesz-e vagy sem.

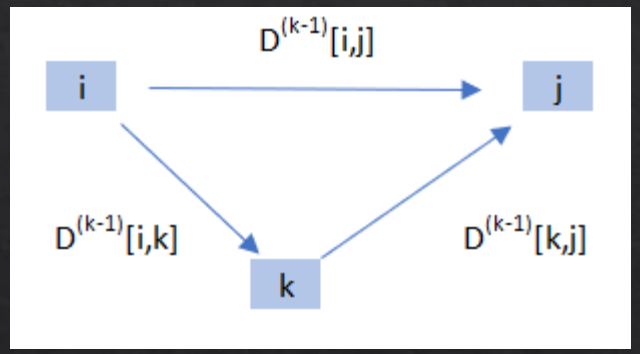


Floyd-Warshall algoritmus

- D_{ij} rekurzív számítása:

$$D_{ij}^{(0)} = \begin{cases} 0, \text{ ha } i = j \\ w(i,j), \text{ ha } (i,j) \in G.E \wedge i \neq j \\ \infty, \text{ ha } i \neq j \wedge (i,j) \notin G.E \end{cases}$$

$$D_{ij}^{(k)} = \begin{cases} D_{ik}^{(k-1)} + D_{kj}^{(k-1)}, \text{ ha } D_{ij}^{(k-1)} > D_{ik}^{(k-1)} + D_{kj}^{(k-1)} \\ D_{ij}^{(k-1)} \text{ különben} \end{cases}$$



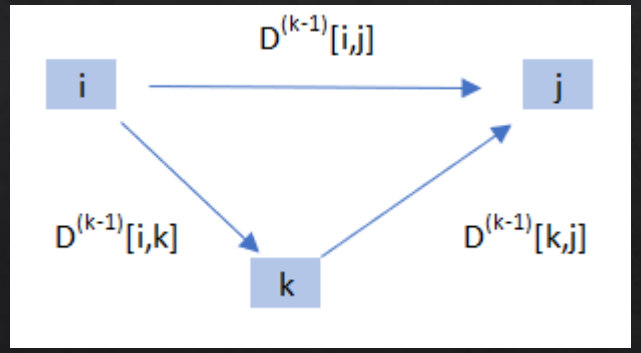
- D^0 lényegében a gráf csúcsmátrixos ábrázolása.
- Amikor k=n, akkor kész a megoldás.

Floyd-Warshall algoritmus

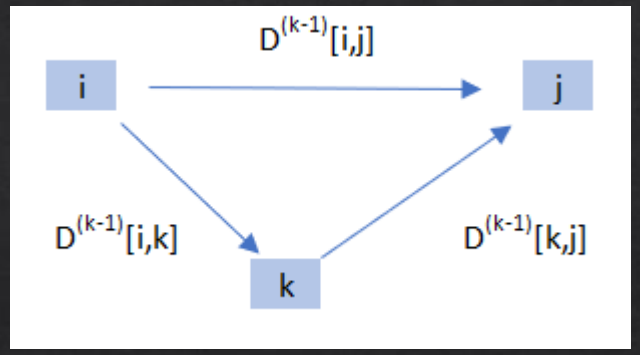
- Π_{ij} rekurzív számítása:

$$\pi_{ij}^{(0)} = \begin{cases} 0, \text{ ha } i = j \\ i, \text{ ha } (i,j) \in G.E \wedge i \neq j \\ 0, \text{ ha } i \neq j \wedge (i,j) \notin G.E \end{cases}$$

$$\pi_{ij}^{(k)} = \begin{cases} \pi_{kj}^{(k-1)}, \text{ ha } D_{ij}^{(k-1)} > D_{ik}^{(k-1)} + D_{kj}^{(k-1)} \\ \pi_{ij}^{(k-1)} \text{ különben} \end{cases}$$

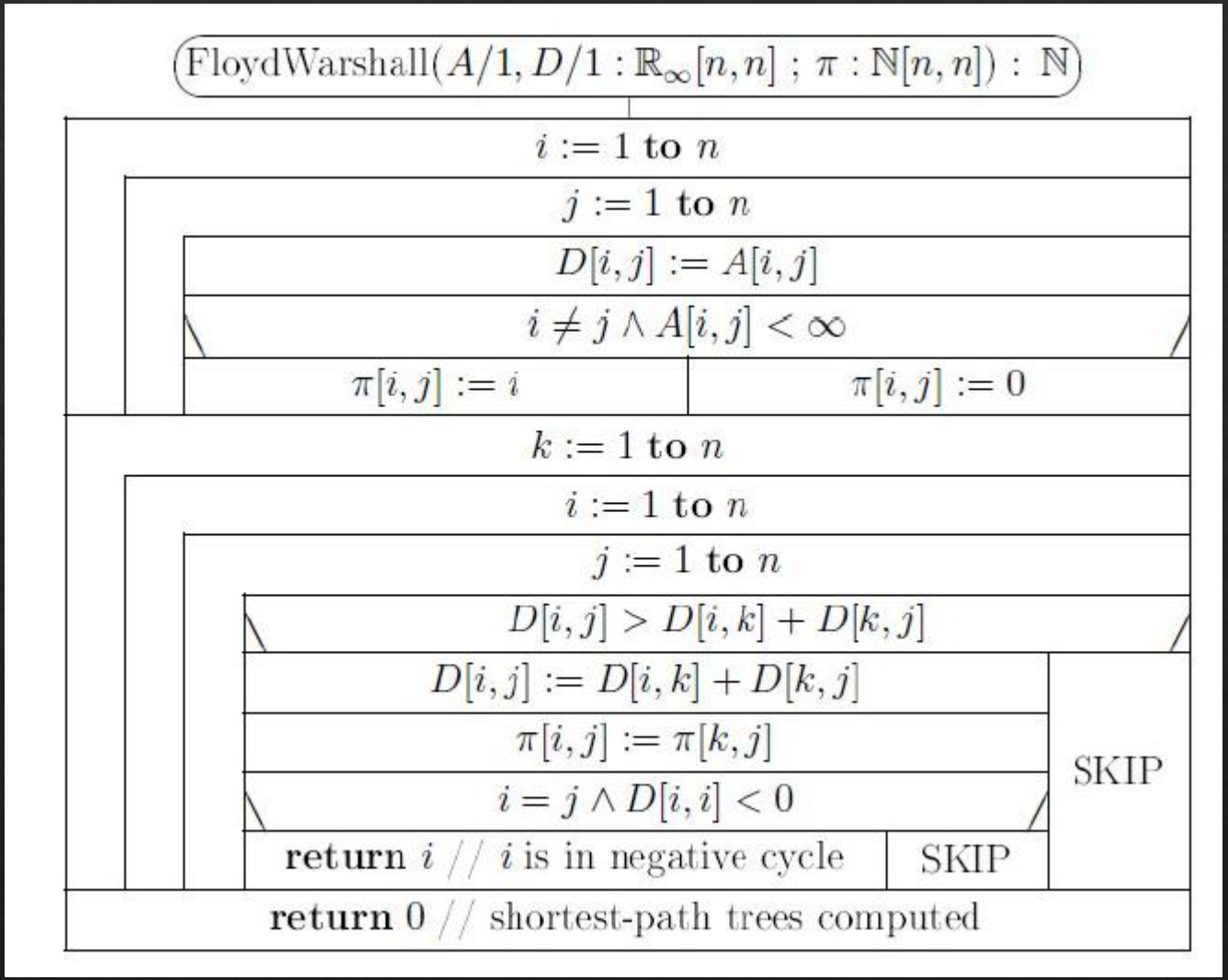


Floyd-Warshall algoritmus



- Az algoritmus mátrixpárok sorozatát állítja elő 0-tól n-ig : n + 1 darab mátrix.
- Az algoritmus működése során mégis elegendő egyetlen D és Π mátrixot fenntartani, mivel a k-adik sor és k-adik oszlop a k - 1 és k-adik lépés között biztosan nem változik meg, hiszen a k épp az út egyik végpontja, így k belső csúcsként az úton nem szerepelhet, tehát D_{ik} és D_{kj} nem változhat meg!
- Floyd-Warshall algoritmus esetén, ha a gráf tartalmaz negatív összsúlyú kört, akkor a főátlóban negatív értékek jelennek meg, ezt az algoritmus jelezni fogja.

Floyd-Warshall algoritmus

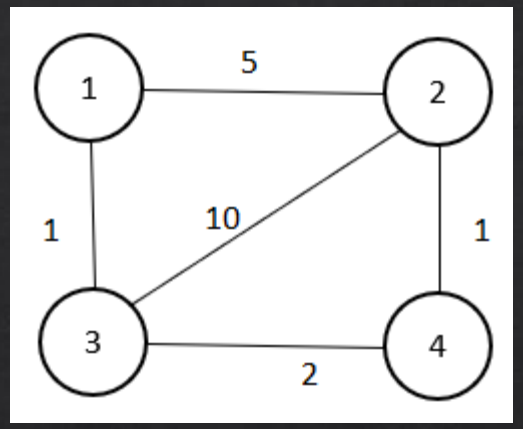


$\Theta(n^2)$

$O(n^3)$

Floyd-Warshall algoritmus lejátszása

- Játszuk le az algoritmust!
- FW_peldak.xlsx



- A végeredmény:

D	1	2	3	4
1	0	4	1	3
2	4	0	3	1
3	1	3	0	2
4	3	1	2	0

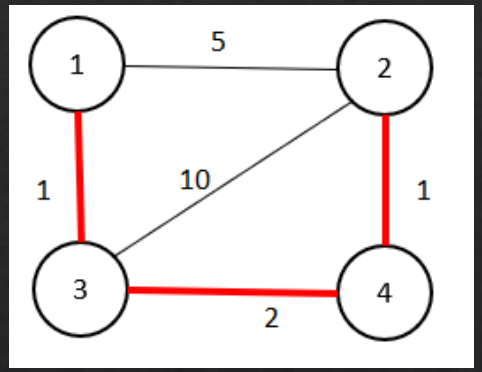
Π	1	2	3	4
1	0	4	1	3
2	3	0	4	2
3	3	4	0	3
4	3	4	4	0

Floyd-Warshall algoritmus lejátszása

- Mit ad meg a végeredmény?
- 1.sor: 1-es csúcsból induló legrövidebb utak:

D	1	2	3	4
1	0	4	1	3
2	4	0	3	1
3	1	3	0	2
4	3	1	2	0

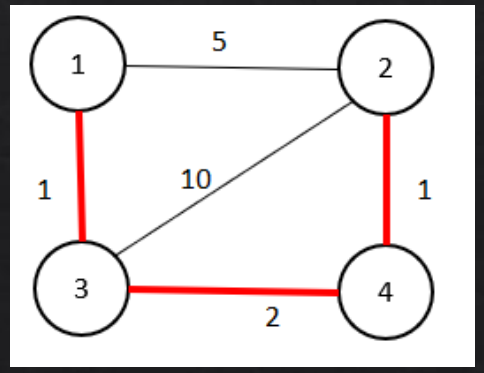
Π	1	2	3	4
1	0	4	1	3
2	3	0	4	2
3	3	4	0	3
4	3	4	4	0



- 2. sor: 2-es csúcsból induló legrövidebb utak:

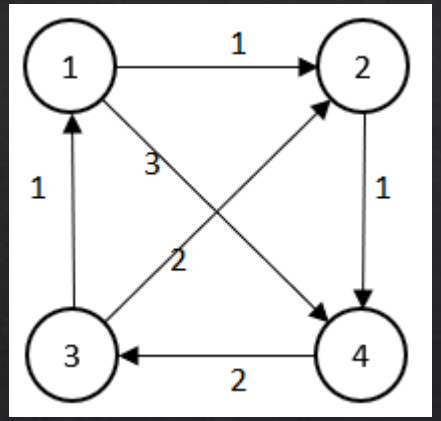
D	1	2	3	4
1	0	4	1	3
2	4	0	3	1
3	1	3	0	2
4	3	1	2	0

Π	1	2	3	4
1	0	4	1	3
2	3	0	4	2
3	3	4	0	3
4	3	4	4	0



Önálló feladat

- Oldjuk meg önállóan a FW_peldak.xlsx található irányított gráfra is a feladatot!



Folytatás: Irányított gráf tranzitív lezártja (következő gyakorlat).